Constructors

A constructor in C++ is a special member function that is automatically called

when an object of a class is created.

A constructor will have exact same name as the class and it does not have any

return type at all, not even void. Constructors can be very useful for setting initial

values for certain member variables.

Example :

```
class Line
{
float length;
public:
void setLength(float len)
{
length=len;
}
void printLength()
{
cout<<”length of the line:”<<length<<endl;
}
/********** Constructor ********************/
Line()
{
length=3.4;
```

Vector India

```
}
};
main()
{
        Line obj; // Create an object of Line
        obj.printLength();
}
```

Output :

3.4

Key Points:

• Constructor name and class name should be same.

• Constructor should not have return type, not even void should be mentioned.

• Constructors should be declared under public access specifiers.

Types of Constructors

1. Default constructor: Constructor without input arguments is called Default

constructor.

An Uninitialized object always invokes default constructor.

Example : Line obj1; // uninitialized object invokes default constructor

2. Parameterised constructor : Constructor with input arguments is called

Parameterised constructor.

Example : Line obj2(4.7f) ; // object with parameters invokes parameterised

constructor.

Example :

Vector India

```
class Line
{
float length;
public:
void setLength(float len)
{
length=len;
}
void printLength()
{
cout<<"length of the line:"<<length<<endl;
}
/********** Constructor ********************/
Line()
{
length=3.4;
}
/*************parameterised constructor ******************/
Line(float f)
{
length=f;
}
};
main()
{
```

Vector India

```
Line obj1; //Created uninitialized object and it invokes the default constructor

obj1.printLength();

Line obj2(5.6f); // created an object with parameters it invokes the

parameterised constructor

}
```

Output :

```
3.4

5.6
```

3. Copy constructor : Constructor with input arguments as same class object

reference is called copy constructor.

Or

A copy constructor is a member function which initializes an object using another

object of the same class. A copy constructor has the following general function

prototype:

ClassName (const ClassName &old_obj)

Example :

```
class Line

{

float length;

public:

void setLength(float len)

{

length=len;
```

Vector India

```
}
void printLength()
{
cout<<”length of the line:”<<length<<endl;
}
/********** Constructor ********************/
Line()
{
length=3.4;
}
/**************parameterised constructor ********************/
Line(float f)
{
length=f;
}
/************** Copy Constructor ******************/
Line(Line &obj)
{
length=obj.length;
}
};
main()
{
Line obj1; //Created uninitialized object and it invokes the default constructor
obj1.printLength();
```

Vector India

Line obj2(5.6f); // created an object with parameters it invokes the

parameterised constructor

obj2.printLength();

Line obj3=obj1; // created an object by assigning existing object, so it invokes

the copy constructor

obj3.printLength();;

}

Output :

3.4

5.6

3.4

Note : Constructor supports function overloading.

In C++, a Copy Constructor may be called in following cases:

1. When an object of the class is returned by value.

2. When an object of the class is passed (to a function) by value as an argument.

3. When an object is constructed based on another object of the same class.

4. When the compiler generates a temporary object.

Shallow copy Vs Deep copy

Shallow Copy:

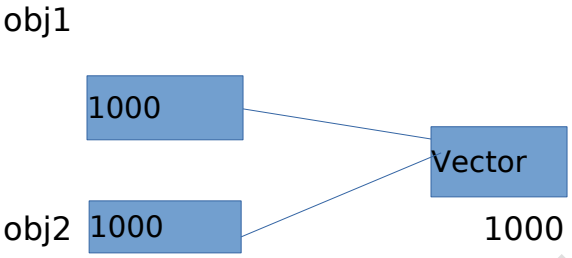
In shallow copy coping is performed member-wise copy between the objects.

This works fine if data members are statically allocated ones, but if data member is

a pointers or any runtime allocation then both the objects pointers points to the same location as show in below diagram, if one object pointer contents

Vector India

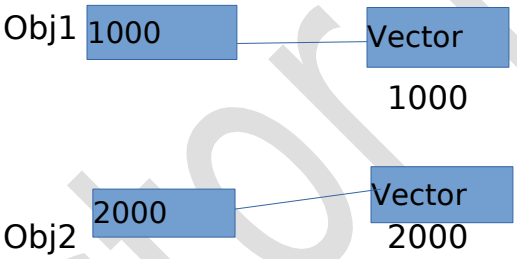
are modified it reflects on other object pointer contents. To avoid this programmer has to supply the copy constructor with deep copy.



Deep Copy:

In Deep copy we allocate separate memory to the pointer and copies the data

pointed by pointer rather than pointer itself as shown in below diagram.



Constructors Vs Member function?

A constructor is different from normal functions in following ways:

- Constructor has same name as the class itself
- Constructors don't have return type
- A constructor is automatically called when an object is created.
- If we do not specify a constructor, C++ compiler generates a default constructor for us (expects no parameters and has an empty body).

Vectorindia